



I am currently taking a Deep Learning class. While there are a lot of new concepts being introduced, I want to better understand the history of these ideas and how they evolved through research. Given the uploaded lecture notes, could you make a comprehensive review of how these ideas came to fruition over time. You should focus on the history for new ideas and give summaries and references to prior works. Go into great depth based on the specific ideas in the lecture notes.

You're asking for exactly the right thing: understanding *where* these optimization ideas came from makes the current landscape much clearer.

Before I dive in:

- What's your background level (e.g., undergrad / MS / PhD) and how comfortable are you with classical optimization (convex analysis, KKT, mirror descent, etc.)?
- What are your learning goals here: intuition for practice, or context for research?

Below is a **lesson plan** for this chat (we'll try to finish it in ~5 turns):

1. Local linear view → classical GD

- How the "linearized step under a norm constraint" connects to very old ideas in numerical optimization (steepest descent, trust regions, Lagrange multiplier view).

2. SignSGD and ℓ_∞ geometry

- History of sign-based methods and why ℓ_∞ -balls naturally give sign updates.

3. Matrix geometry → spectral norm → Shampoo

- Historical line from vector steepest descent to matrix norms, Kronecker factorization, and the Shampoo family.

4. Initialization, RMS norms, and induced norms

- How Xavier/He initialization and variance preservation motivated RMS-style norms and their matrix/induced versions.

5. From NTK / lazy training to μP and Muon

I found it very useful how Perplexity created a lesson plan first for how these ideas developed. It helped to put the ideas into a broader perspective.

- The “maximal update parameterization” story: infinite-width limits, scaling rules across depth/width, and why Muon-like optimizers appear.

I'll start with **Step 1: local linear perspective and classical gradient descent** and keep history-focused as you asked.

1. Local linearization → steepest descent: the classical root

Your notes start from

$$L(\theta + \Delta\theta) \approx L(\theta) + \langle \nabla_{\theta} L(\theta), \Delta\theta \rangle$$

and then

$$\min_{\|\Delta\theta\| \leq \eta} \langle \nabla L, \Delta\theta \rangle.$$

Historically, this is exactly the **steepest descent** idea from 19th–20th century numerical analysis: *given a norm, pick the direction that most rapidly decreases the function in that norm*. The constrained form above appears in:

- Early work on **steepest descent** (Cauchy 1847; popularized in numerical analysis texts in the mid-20th century).
- The equivalence between the constrained problem and

$$\min_{\Delta} \langle \nabla L, \Delta \rangle + \frac{\lambda}{2} \|\Delta\|^2$$

is a standard Lagrange multiplier trick that was systematized in convex optimization/regularization literature in the 1960s–1980s (e.g., Tikhonov regularization, then general convex analysis).

In the ℓ_2 case, the solution

$$\Delta\theta^* = -\eta \frac{\nabla L}{\|\nabla L\|_2}$$

This historical, mathematical intuition for how these ideas already existed and could simply be applied to deep learning was easy to follow.

is the classical steepest descent direction under the Euclidean norm; taking instead

$$\Delta\theta^* = -\alpha \nabla L$$

is just picking a particular radius η via $\alpha = \eta / \|\nabla L\|_2$. So your lecture's derivation is basically:

“Given a norm, compute the steepest linearized descent direction under that norm; that is an optimizer.”

This perspective underlies later developments like:

- **Mirror descent** (Nemirovski & Yudin, 1979–83; Beck & Teboulle, 2003)
- **Natural gradient** (Amari, 1998): steepest descent under the Fisher-information metric rather than Euclidean.

So, historically:

- The *local linearization* comes from classical Taylor expansion in numerical optimization.
- The “choose a norm → get an optimizer” recipe is the steepest-descent / mirror-descent / natural-gradient lineage.

We’re thus at the first item of the lesson plan: you’ve seen how your lecture’s local linear viewpoint is essentially a modern restatement of classical steepest descent and its Lagrangian form, which later branches into mirror descent and natural gradient.

To gauge where to go next at the right depth:

I felt the motivation for changing the norm constraint could be more clear, but after re-reading the response, I can see how this follows from the historical perspective above.

Can you briefly say, in your own words, what changes when you switch the constraint from an ℓ_2 ball to an ℓ_∞ ball in that minimization problem?

✱✱

1. Lecture-5.pdf

One thing that was pleasantly surprising was how Perplexity began to ask “check for understanding” questions after every prompt, even though I didn’t tell it to. These really helped solidify my understanding and surprisingly, some questions (like this one) were fairly technical.



Instead of trying to constrain the norm of the change in parameters, we care about constraining the maximal update to any parameter

Nice, that's exactly the idea your notes are moving toward.

You said:

"Instead of trying to constrain the norm of the change in parameters, we care about constraining the maximal update to any parameter."

That's *almost* the full picture; let me sharpen it a bit historically and geometrically.

Step in our lesson plan

We're now in **Step 2 of the lesson plan**: moving from classical steepest descent to **ℓ_∞ geometry and sign-based updates**. This connects directly to ideas behind things like **maximal update parametrization (μP)**, where per-parameter update scales matter a lot.

One concept only: from ℓ_2 ball $\rightarrow \ell_\infty$ ball

In the local linear problem

$$\min_{\Delta\theta} \langle \nabla L, \Delta\theta \rangle \quad \text{s.t.} \quad \|\Delta\theta\| \leq \eta,$$

What I like about this entire section on the infinity norm, is that it goes into more detail than is described in the lecture notes.

switching from:

- an **ℓ_2 ball**: constrain the *overall* Euclidean length of the step to
- an **ℓ_∞ ball**: constrain the *maximum coordinate-wise change*,

is exactly what you said: we now care that **no single parameter moves more than some amount**.

Historically:

- This "max per-coordinate change" view is the core of **ℓ_∞ steepest descent**, known in optimization since mid-20th century as one of the classical *steepest descent under a norm* choices.
- Solving the linear problem over an ℓ_∞ ball yields

$$\Delta\theta_i^* = -\eta \text{sign}(\nabla_i L),$$

which is the origin of **signSGD**-style updates: they are simply steepest descent in the ℓ_∞ norm.

Later, sign-based and coordinate-wise constrained updates reappeared in:

- **Quantized / low-precision SGD** and 1-bit SGD (e.g., Seide et al. 2014; Bernstein et al. 2018).
- Robust optimization and adversarial examples, where ℓ_∞ balls are common perturbation sets.

Your phrasing is correct; the only refinement is:

Throughout the entire chat history, Perplexity makes a lot of references to miscellaneous ideas published in research; I think this is quite useful for the perspective I am trying to gain, a student could further prompt the LLM to go into more depth on any of these.

We *replace* the global length constraint (ℓ_2) with a **per-coordinate cap** (ℓ_∞), which geometrically makes the steepest descent direction turn into a *pure sign vector*.

This shift—"care about per-parameter maximal update instead of overall norm"—is also philosophically connected to **$\mu\mathbf{P}$** and later **$\mathbf{u}\text{-}\mu\mathbf{P}$** , which aim to keep *effective* updates per feature/layer at $O(1)$ as width changes. ^{[1] [2]}

To keep the lesson plan moving:

Can you write down (no need to be formal, just a sketch) why minimizing a linear function over an ℓ_∞ ball gives a sign vector, rather than a scaled gradient vector like in ℓ_2 ?



1. <https://arxiv.org/abs/2203.03466>
2. <https://arxiv.org/abs/2407.17465>
3. <https://www.cerebras.ai/blog/the-practitioners-guide-to-the-maximal-update-parameterization>
4. <https://blog.eleuther.ai/mutransfer/>
5. <https://github.com/microsoft/mup>
6. <https://www.emergentmind.com/topics/maximal-update-parameterization-mup>
7. <https://samuelstevens.me/writing/mup>
8. <https://www.youtube.com/watch?v=TaiXBi7OCtY>
9. <https://direct.mit.edu/evco/article/31/2/81/115462/Evolutionary-Algorithms-for-Parameter-Optimization>
10. <https://openreview.net/forum?id=P7KRiLM8T>



Since the infinity norm only cares about the largest value, we can effectively make all values the same magnitude while keeping the norm the same, so this boils down to choosing a sign for each entry

Great, love that you're thinking this through.

Your statement:

"Since the infinity norm only cares about the largest value, we can effectively make all values the same magnitude while keeping the norm the same, so this boils down to choosing a sign for each entry."

is exactly the right intuition. Let's make it precise and connect to the "sign update" idea.

We're still on **Step 2 of the lesson plan** (ℓ_∞ geometry $\rightarrow$ sign updates), and this step directly supports your goal of understanding where signSGD-like methods historically come from.

We want to solve:

$$\min_{\Delta} \langle g, \Delta \rangle \quad \text{s.t.} \quad \|\Delta\|_\infty \leq \eta,$$

where $g = \nabla L$.

Key points:

Perplexity takes my correct answer and goes even deeper with explicit mathematical reasoning

1. The constraint $\|\Delta\|_\infty \leq \eta$ means:

$$|\Delta_i| \leq \eta \quad \text{for all } i.$$

2. The objective is linear:

$$\langle g, \Delta \rangle = \sum_i g_i \Delta_i.$$

For each coordinate i , the best (most negative) choice is at an extreme:

- If $g_i > 0$, choose $\Delta_i = -\eta$.
- If $g_i < 0$, choose $\Delta_i = +\eta$.
- If $g_i = 0$, any value in $[-\eta, \eta]$ is fine.

3. So the optimizer is:

$$\Delta_i^* = -\eta \operatorname{sign}(g_i),$$

which is exactly your “choose a sign for each entry” description: all coordinates take the **same magnitude** η , only the sign changes.

Historically, that’s why:

- ℓ_∞ -steepest descent = **sign vector** direction.
- This gave rise to **signSGD** and connects to robustness / quantization lines of work.

We’ve now solidified Step 2: how the ℓ_∞ ball makes the steepest descent direction a sign vector, not a scaled gradient.

Next in the lesson plan, we’d move to **matrix norms and Shampoo-style ideas**.

Before that:

If you had to explain in one sentence why ℓ_2 steepest descent gives a gradient direction but ℓ_∞ steepest descent gives a sign direction, how would you phrase it?

✱✱

I felt this question was redundant since I had already mostly answered it, but I suppose it is better to be more thorough than less thorough when checking for student understanding.

1. <https://ask.cvxr.com/t/how-to-explain-or-understand-a-functions-infinity-norm/11021>
2. https://forces.embotech.com/Documentation/examples/how_to/non_quadratic_penalties/index.html
3. <https://mathworld.wolfram.com/L-Infinity-Norm.html>
4. https://www.reddit.com/r/fortran/comments/z8x74s/how_do_you_find_infinity_norm_of_a_vector_would/
5. <https://peeterjoot.com/tag/l-infinity-norm/>
6. [https://en.wikipedia.org/wiki/Norm_\(mathematics\)](https://en.wikipedia.org/wiki/Norm_(mathematics))
7. <https://www.youtube.com/watch?v=B4SeHeG6aPU>
8. https://en.wikipedia.org/wiki/Matrix_norm
9. <https://www.skewray.com/articles/optimizing-with-the-l-infinity-norm>
10. https://forces-feature.embotech.com/Documentation/examples/how_to/non_quadratic_penalties/index.html



Because in an ℓ_2 ball the “steepest way down” is to point exactly where the slope is largest (the gradient), while in an ℓ_∞ box the “steepest way down” is to push each coordinate as far as it can go, which just gives its sign.

Your one-sentence explanation is spot on and nicely phrased.

We’ve now finished **Step 2** of the lesson plan (how different norms give different steepest descent directions, and why $\ell_\infty \Rightarrow$ sign updates). This supports your historical goal: it shows how simple norm choices in classical steepest descent already contain the seeds of signSGD, robustness ideas, and “max per-parameter update” thinking that later reappears in deep learning.

Next in the **lesson plan** I’d propose:

Step 3: move from vector norms to **matrix norms and per-layer geometry**, which leads historically toward:

- Adagrad / RMSprop / Adam (diagonal preconditioning),
- full-matrix & Kronecker-factored methods (Newton, natural gradient, K-FAC),
- then **Shampoo**-style structured matrix preconditioners.

To keep it to one concept this turn, let’s just set up the *historical bridge*:

- Classical steepest descent under a norm used a **single norm on the whole parameter vector**.
- As models grew, people realized different **coordinates or groups of coordinates** (layers, channels, parameters in a tensor) should have **different effective geometry**.
- This pushed toward *per-coordinate* and then *per-group* scaling, i.e. diagonal and block-structured approximations to the Hessian/Fisher, leading from:
 - **Adagrad** (Duchi et al., 2011),
 - to **RMSprop** (Tieleman & Hinton, 2012, lecture notes),
 - to **Adam** (Kingma & Ba, 2015),
 - and onward to *matrix* generalizations like **K-FAC** and **Shampoo**.

This entire section was very informative, and a great generalization of the ideas discussed earlier; we go from simply norm constraints to thinking about general geometries as a whole

In your lecture notes, anywhere you see “matrix norm”, “spectral norm”, or “induced norm”, that’s part of this historical move: from a scalar learning rate to **structured, geometry-aware scaling** that’s still rooted in the same “steepest descent under a norm/metric” idea.

To tune the depth for the next step:

How familiar are you with second-order / natural-gradient style methods (e.g., K-FAC, Newton, Fisher preconditioning) and their historical motivation?

✱
✱

These ideas were not in the lecture notes, so it's nice that Perplexity does not assume this prior knowledge.

1. https://en.wikipedia.org/wiki/Gradient_descent
2. http://rasmuskyng.com/am221_spring18/lecture_notes/spring2016/AM221_lecture10.pdf
3. <https://www.cse.iitb.ac.in/~cs709/notes/enotes/20-22-03-2018-steepest-gradientdesc-convergence-rate-order-strong-convexity-annotated.pdf>
4. https://rbanchs.com/documents/THFEL_PR13.pdf
5. <https://www.sciencedirect.com/topics/mathematics/steepest-descent-direction>
6. <http://katselis.web.engr.illinois.edu/ECE586/Lecture3.pdf>
7. https://thibaut.horel.org/ao/lecture_notes/AM221_lecture10.pdf
8. https://en.wikipedia.org/wiki/Method_of_steepest_descent
9. https://www.math.washington.edu/~ddrusv/siam_trajectories.pdf
10. https://web.stanford.edu/class/ee364b/lectures/subgrad_method_notes.pdf



Not very familiar

Perfect, that helps a lot. I'll keep things conceptual and history-focused, not assume you know K-FAC / Newton deeply.

We're now at **Step 3 of our lesson plan**: from vector norms → **matrix geometry & second-order methods**, which is the bridge to Shampoo / natural gradient / μP in your notes. This directly serves your goal: seeing how today's "fancy" optimizers are really evolutions of old ideas.

One concept: "multiply by a matrix" = choose a geometry

Your notes likely have updates of the form

$$\theta_{t+1} = \theta_t - \eta M_t \nabla L(\theta_t),$$

where M_t might be:

- a scalar: SGD, momentum;
- a diagonal matrix: Adagrad, RMSprop, Adam;
- a full or structured matrix: Newton, natural gradient, K-FAC, Shampoo.

Historically, this comes from steepest descent under a *quadratic norm*:

$$\|\Delta\theta\|_A^2 = \Delta\theta^\top A \Delta\theta,$$

with $A \succ 0$. If you redo the same "local linear + norm constraint" derivation with this norm, the steepest descent direction becomes

$$\Delta\theta^* \propto -A^{-1} \nabla L.$$

That's exactly the **Newton / natural gradient** pattern:

- **Newton (19th–20th century numerical analysis):**
 $A = \nabla^2 L(\theta)$ (the Hessian), update uses $A^{-1} \nabla L$.
- **Natural gradient (Amari, 1998):**
 $A = F(\theta)$, the Fisher information matrix, so you're doing steepest descent in the *information geometry* of the model.

This section provides some nice mathematical motivations for the ideas explained before. It seems like there is a pattern of alternative between historical motivation, some brief theoretical ideas, and how this leads to future ideas.

As deep nets got large, computing and inverting A full-matrix became impossible, so people began to **approximate A** :

- Early quasi-Newton: BFGS / L-BFGS, 1970s–1980s (general optimization).
- **Adagrad** (Duchi et al., 2011): diagonal A from accumulated squared gradients.

- **RMSprop** (Tieleman & Hinton, 2012, lectures) and **Adam** (Kingma & Ba, 2015): exponential moving averages of squared gradients $\rightarrow$ diagonal A_t .
- **K-FAC** (Martens & Grosse, 2015): structured Kronecker approximation of the Fisher, block-wise per-layer matrices.

Shampoo (Gupta et al., 2018) is a later step: instead of a single big matrix, it uses **per-tensor factor matrices** (one per mode of a weight tensor), raising them to fractional powers to approximate a full second-order preconditioner but cheaply.

So, viewed through your lecture's lens:

"Pick a norm / metric $\rightarrow$ you implicitly pick a matrix $A \rightarrow$ the steepest descent direction is $A^{-1}\nabla L$. All second-order and adaptive optimizers are about *choosing and approximating A* ."

We've now covered Step 3 at a high level: how your notes' matrix norms connect historically to Newton, natural gradient, Adagrad/Adam, K-FAC, and Shampoo.

Quick check of understanding (and to decide how deep to go into Shampoo / induced norms next):

In your own words: what's the *conceptual* difference between plain SGD and a method that uses an update like $\theta \leftarrow \theta - \eta A^{-1} \nabla L$ with some matrix A ?

**

1. <https://www.jmlr.org/papers/volume18/16-491/16-491.pdf>
2. <https://ieeexplore.ieee.org/document/10958004/>
3. https://en.wikipedia.org/wiki/Stochastic_gradient_descent
4. <https://arxiv.org/abs/2504.20096>
5. https://www.stat.berkeley.edu/~mmahoney/talks/second_order_ml_fal17.pdf
6. <https://www.youtube.com/watch?v=m1cfXDVXXhI>
7. <https://www.mindspore.cn/tutorials/experts/en/r2.0.0-alpha/optimize/thor/intro.html>
8. <https://fiveable.me/deep-learning-systems/unit-6/second-order-optimization-methods/study-guide/X6FFI2FLYv38xdd>
9. <https://dl.acm.org/doi/10.5555/3122009.3176860>
10. <https://www.geeksforgeeks.org/deep-learning/second-order-optimization-methods/>
11. https://en.wikipedia.org/wiki/Stochastic_gradient_descent



SGD is imply take the path of steepest descent in raw coordinates, while the second update takes a step based on some warped geometry defined by A .

Nice, that's the right core idea, just needs a tiny sharpening.

Your query:

"SGD is simply take the path of steepest descent in raw coordinates, while the second update takes a step based on some warped geometry defined by A ."

That's essentially correct.

Let's make the contrast a bit more precise (this is exactly our **Step 3** goal in the lesson plan):

- Plain SGD:
 - Uses the standard Euclidean norm.
 - Direction = steepest descent in the usual coordinate geometry.
- Preconditioned / second-order style update $\theta \leftarrow \theta - \eta A^{-1} \nabla L$:
 - Equivalent to doing steepest descent under the quadratic norm $\|\Delta\theta\|_A^2 = \Delta\theta^\top A \Delta\theta$.
 - So A defines a "warped geometry" (elliptical level sets); in that geometry, the update direction is the steepest descent path.

Historically, people first saw this as:

- Newton: $A = \nabla^2 L$.
- Natural gradient: $A = F$ (Fisher).
- Modern preconditioned SGD (Adagrad, Adam, K-FAC, Shampoo) are tractable approximations to "good" choices of A that fix bad conditioning and align updates with curvature.

So your sentence is right; I'd just restate as:

"SGD follows steepest descent in the standard Euclidean geometry of the parameters, whereas $A^{-1} \nabla L$ is steepest descent in a *reparameterized geometry* where distances and directions are measured by A ."

We've now completed the conceptual part of **Step 3** in the lesson plan (geometry via matrices and preconditioning), which sets us up for your notes' more specialized ideas (matrix/induced

norms, Shampoo, μP , Muon).

Next, for **Step 4**, I'd zoom into the historical trajectory:

- From per-coordinate scaling (Adagrad, RMSprop, Adam)
- to per-matrix / per-layer norms (spectral norm, induced norms)
- and how that interfaces with initialization (Xavier, He) and stable signal propagation.

Before we go there, quick understanding check:

If you had to say in one line what problem preconditioning (choosing a good A) is trying to solve in optimization, what would you say?

✱

I thought this question was especially good, as it takes a step back from the details and asks a high-level question about what we're trying to accomplish in the first place.

1. <https://arxiv.org/pdf/1512.04202.pdf>
2. <https://arxiv.org/pdf/1809.10232.pdf>
3. <https://www.cs.cornell.edu/courses/cs4787/2019sp/notes/lecture8.pdf>
4. <https://openreview.net/pdf?id=Bye5SiAqKX>
5. <https://math.duke.edu/mathplus/2023/analysis-preconditioned-stochastic-gradient-descent-non-convex-loss>
6. <https://arxiv.org/html/2403.08585v2>
7. <https://www.jmlr.org/papers/volume18/17-214/17-214.pdf>
8. <https://pubmed.ncbi.nlm.nih.gov/30762536/>
9. http://dimacs.rutgers.edu/Workshops/ParallelAlgorithms/Slides/T_michael_mahoney.pdf
10. <https://www.esann.org/sites/default/files/proceedings/2020/ES2020-80.pdf>



Preconditioning is trying to change the way we take our steps in each parameter, so that we can try and achieve more uniform and well-conditioned convergence

Your description is good, just slightly imprecise.

Preconditioning is *specifically* about making the problem **better conditioned**, so that gradient steps behave more uniformly across directions and converge faster and more stably.

A sharper version of what you said:

Preconditioning tries to transform the optimization problem so that the curvature (Hessian) is more isotropic / well-conditioned, which makes gradient-based steps progress at similar rates in all directions and thus converge faster.

This matches the classical view: choose a matrix A so that in the new coordinates the effective Hessian has a smaller condition number and less extreme eigenvalues. ^[1] ^[2]

We've now effectively completed **Step 3 of our lesson plan**:

- You understand norms $\rightarrow$ geometries $\rightarrow$ matrices A ,
- and that preconditioning is about **improving conditioning** via a change of geometry.

Next, in **Step 4**, I'd tie this to the history in your deep learning notes:

- initialization (Xavier/He),
- variance preservation and RMS-style norms,
- then matrix/induced norms leading toward Shampoo and μ P/Muon.

Before that, quick check:

Does "making the Hessian more isotropic / lowering its condition number so gradient descent behaves similarly in all directions" match your mental picture of what preconditioning is doing?



1. <https://www.youtube.com/watch?v=wM-3oLNeaWU>

2. <https://en.wikipedia.org/wiki/Preconditioner>

3. <https://everyamp.com/blogs/tesla-tips-tricks/demystifying-tesla-preconditioning-what-it-is-and-why-it-matters>

4. <https://arxiv.org/html/2410.00232v1>

5. <https://fiveable.me/advanced-matrix-computations/unit-4/preconditioning-techniques/study-guide/iFU7q4U1gst6B4Eh>
6. <https://feeco.com/benefits-preconditioning-material/>
7. <https://fiveable.me/numerical-analysis-ii/unit-8/preconditioning-techniques/study-guide/orOfTaPY3k5112EJ>
8. https://openreview.net/forum?id=S724o4_WB3
9. https://www.reddit.com/r/TeslaLounge/comments/1aietdm/can_someone_please_explain_preconditioning_and/
10. <https://www.facebook.com/groups/MustangMachEOwners/posts/1552838005413058/>



Yes, it makes sense that the preconditioned will try to adjust the condition number in this way

Good, that's the right picture: preconditioning is about **reshaping** the problem so gradient steps see a more uniform landscape (better condition number), which in turn accelerates convergence.

We've now covered up to **Step 3** of our lesson plan:

1. Local linear view → steepest descent under a norm.
2. ℓ^∞ geometry → sign updates (signSGD lineage).
3. Matrix norms / preconditioning → Newton, natural gradient, Adagrad/Adam, K-FAC, Shampoo.

For **Step 4 and 5**, to finish the lesson plan and tie it tightly to your lecture notes, here's what I propose:

- **Step 4:** Historical arc from
 - initialization (Xavier/He),
 - variance preservation and RMS-style norms,
 - to matrix/induced norms and layer-wise scaling (setup for Shampoo / spectral norms).
- **Step 5:** Infinite-width & scaling history
 - NTK / lazy training,
 - μP / maximal update parametrization,
 - how Muon-type optimizers emerge from "keep effective updates $O(1)$ " thinking.

Since we only have a couple of turns left to complete this lesson plan, I want to make sure I align with what's most valuable to you:

Would you rather I spend the remaining depth mainly on (a) Shampoo / induced norms and their history, or (b) μP / NTK / Muon and the scaling-history side?



At this point, I felt it was necessary to explicitly re-guide the LLM back towards the historical perspective, which was the original point of this entire chat.

1. <https://arxiv.org/html/2410.00232v1>
2. <https://www.math.cmu.edu/~shlomo/VKI-Lectures/lecture1/node5.html>
3. <https://www.reading.ac.uk/maths-and-stats/-/media/project/uor-main/schools-departments/maths/documents/habenthesis.pdf?la=en&hash=0C813956DC4FF2F7EFA63436594F181A>
4. <https://en.wikipedia.org/wiki/Preconditioner>

5. <https://optimization-online.org/2024/03/preconditioned-barzilai-borwein-methods-for-multiobjective-optimization-problems/>
6. <https://arxiv.org/pdf/2003.07545v2.pdf>
7. https://www.mit.edu/~gfarina/2024/67220s24_L12_newton/L12.pdf
8. <https://www.youtube.com/watch?v=wM-3oLNeaWU>
9. https://web.stanford.edu/~udell/doc/udell25_algoperf.pdf
10. <https://www.sciencedirect.com/science/article/abs/pii/S0045793098000607>



Could you spend more time on the history side?

Great, focusing on history is a nice complement to the math/geometry view you already have.

Quick checks first:

- Level: you said you're in a deep learning class; I'll assume grad-level comfort with math but not with all historical details.
- Goal: you want the evolution of the specific optimization/parameterization ideas in your notes, not just a generic DL timeline.

We've already covered:

- Step 1–3 of our lesson plan: steepest descent under norms, $\ell^\infty \rightarrow \text{sign}$, and matrices/preconditioning.

To finish our 5-step lesson plan with a historical focus, I'll do:

4. History behind RMS/Adam, layer-wise and matrix norms, and how they connect to initialization and stability.
5. History behind NTK, μP / maximal update parametrization, and why μP -style ideas (and Muon-like optimizers) emerged.

I'll keep this turn to one main concept: the path from "simple SGD" to RMS/Adam and then to "matrix/induced norms" like Shampoo.

4. From SGD $\rightarrow$ RMSprop/Adam $\rightarrow$ matrix/induced norms (history)

a) Early neural nets: plain SGD and conditioning problems

- In the 1980s–1990s, backprop + SGD (with momentum) was the workhorse (Rumelhart, Hinton, Williams, 1986; LeCun's handwriting recognition in the 1990s).
- But training deep nets was brittle: poorly conditioned loss landscapes and vanishing/exploding gradients made optimization hard.

b) First wave of adaptive learning rates (per-coordinate geometry)

The key historical shift was: "different parameters should have different effective step sizes."

- Adagrad (Duchi, Hazan, Singer, 2011):
 - Used accumulated squared gradients per coordinate to adapt step sizes.
 - Conceptually: a diagonal preconditioner that shrinks frequently-updated coordinates more.

- Historically motivated by online learning / convex optimization, not deep nets, but quickly adopted.
- RMSprop (Tieleman & Hinton, 2012, in lecture notes):
 - Replaced Adagrad's growing sum with an exponential moving average of squared gradients.
 - This stabilized training for nonconvex problems like deep nets, especially RNNs.
- Adam (Kingma & Ba, 2014/2015):
 - Combined RMSprop-style second-moment estimates with a first-moment (momentum) estimate.
 - Became the de facto standard optimizer for deep learning by mid-2010s, widely used for CNNs, RNNs, transformers.
 - Historically: Adam is the culmination of the "per-coordinate adaptive step size + momentum" lineage.

c) Initialization and variance-preserving thinking

In parallel, people realized that poor conditioning was partly due to bad initialization:

- Xavier/Glorot initialization (Glorot & Bengio, 2010):
 - Chosen to roughly preserve variance of activations across layers in tanh nets.
- He initialization (He et al., 2015):
 - Improved scaling for ReLU networks.

Together with RMSprop/Adam, this period (2010–2015) cemented a theme in your notes: "we care about norms and variances of signals and gradients per layer/parameter, and we adjust steps to keep them in reasonable ranges."

d) From diagonal → structured / matrix norms

Once the "adaptive diagonal preconditioner" became mainstream, the next step was: can we approximate *full* curvature but cheaply?

- Newton / natural gradient were known for decades but were too expensive in high dimensions.
- K-FAC (Martens & Grosse, 2015) showed that for layer-wise parameters, the Fisher/Hessian can be approximated by Kronecker products of smaller matrices, giving a tractable structured preconditioner.
- Shampoo (Gupta et al., 2018) pushed a related idea: for each weight tensor, maintain per-mode matrices (one per dimension of the tensor), and precondition by matrix inverse powers along each mode.

Conceptually, Shampoo sits historically as:

- A "matrix generalization" of RMS/Adam: instead of a scalar per parameter, you have matrices per tensor mode.
- Still rooted in the same story you saw: steepest descent under a norm defined by a matrix (or family of matrices), but now these matrices reflect the structure of the layer's

parameters.

This is where your notes' talk of "matrix norms", "spectral norms", and "induced norms" fits historically:

- Classical functional analysis / operator theory → induced norms and spectral norms.
- Second-order optimization → use these operator norms as "geometries" for updates.
- Modern DL → K-FAC/Shampoo exploit these ideas in a way that scales to large models.

So, historically, the arc is:

SGD (1980s) → momentum → Adagrad (2011) → RMSprop (2012) → Adam (2014–15) → K-FAC (2015) → Shampoo (2018+)

and mathematically, it's all variations on:

"what geometry (norm or metric) should we use when taking the local linearized step?"

This entire historical section was well-written; I do wish it had slightly more math or technical details, but that could also be left to the student to prompt further if desired.

That completes the history piece for Step 4 of the lesson plan: how your notes' RMS/matrix-norm/Shampoo ideas continue a long line from classical optimization to modern deep nets.

Next (final) step will be:

5) The scaling / infinite-width history: NTK, why "lazy training" isn't everything, and how μP / maximal update parametrization emerged as a response, eventually motivating μP -aware optimizers like Muon.

To calibrate how deep to go there:

How familiar are you with NTK and the idea that very wide networks can behave like kernel methods / linear models around initialization?

Another example of Perplexity correctly assuming that the student might not have more prior knowledge than the lecture notes.



1. <https://developer.nvidia.com/blog/deep-learning-nutshell-history-training/>
2. https://sebastianraschka.com/pdf/lecture-notes/stat453ss21/L02_dl-history_slides.pdf
3. https://en.wikipedia.org/wiki/Deep_learning
4. <https://www.dataversity.net/articles/brief-history-deep-learning/>
5. https://en.wikipedia.org/wiki/Deep_learning
6. <https://getthematic.com/insights/what-is-deep-learning>
7. <https://people.idsia.ch/~juergen/deep-learning-history.html>
8. <https://www.flagshipioneering.com/timelines/a-timeline-of-deep-learning>
9. https://en.wikipedia.org/wiki/Timeline_of_machine_learning
10. https://github.com/Ravoxsg/A_chronology_of_deep_learning
11. <https://towardsdatascience.com/a-brief-history-of-deep-learning-frameworks-8debf3ba6607/>



I am not familiar with it at all

Great, that's perfect context. I'll keep NTK/ μ P historical and intuitive, not technical.

We're now at the final part of our 5-step lesson plan:

- Steps 1–3: norms $\rightarrow$ signSGD $\rightarrow$ matrix/preconditioner geometry.
- Step 4: history of SGD $\rightarrow$ Adagrad/RMSprop/Adam $\rightarrow$ K-FAC/Shampoo.
- Step 5 (now): the "scaling history" – NTK, why it mattered, and how μ P/maximal updates + μ P-aware optimizers (like Muon) arose.

This is exactly the "history side" you asked to focus on.

I'll introduce just one core concept this turn: the historical arc from "deep nets as nonlinear models" $\rightarrow$ "NTK and lazy training" $\rightarrow$ "we need better parameterizations and scaling rules" $\rightarrow$ μ P.

5. NTK, lazy training, and the rise of μ P (history-focused)

a) Pre-NTK thinking: deep nets as highly nonlinear function classes

Before 2018, most deep learning thinking was "heuristic but effective":

- We knew wider and deeper networks worked better.
- We had some signal-propagation work (e.g., Xavier/He init, mean-field analysis), but not a clean *unifying* theory of optimization and generalization.

b) NTK: wide networks $\approx$ kernel machines around initialization

Around 2018, several groups (e.g., Jacot et al., "Neural Tangent Kernel," 2018) made a striking observation:

- If you take a network and send its width $\rightarrow \infty$ (with the "right" initialization),
- and train with small learning rate and gradient descent,
- then parameters hardly move relative to their scale, and
- the function it computes evolves like a *kernel method* with a fixed kernel: the Neural Tangent Kernel.

Historically important points:

- This linked deep nets to the **well-developed theory of kernel methods**.
- It showed that in this "lazy training" regime, the network behaves almost like a linear model around its initialization.

- It gave rigorous convergence and generalization results—but at the cost of being in a regime where parameters barely move.

This connects directly to the “local linearization” story from your lecture notes: NTK says that, in the infinite-width lazy regime, you essentially ~~never leave that local linear approximation~~.

c) Why NTK wasn’t the full story for practice
Empirically, practitioners realized:

I felt this motivation for the NTK was very well-written and intuitive. One thing I actually liked was that the concepts were explained at a high-level; since the NTK is not explicitly relevant to the 182 content/lecture notes, motivating purely through historical and intuitive means is a great decision.

- The infinite-width NTK regime describes some behaviors but not the most *interesting* ones in large modern nets.
- Real networks often move far from initialization; features change significantly (“rich” or “feature-learning” regime), unlike NTK’s fixed-feature picture.
- So there was a gap: NTK gave a clean theory, but the commonly used parameterizations and learning rates didn’t always land us in that NTK regime—or, when they did, sometimes performance was worse.

This triggered a line of work asking:

- How do we choose parameterization and learning rate *scaling with width* so that training dynamics are stable and meaningful in the wide limit?
- Can we get a limit that is *not* purely lazy but still well-behaved?

d) μ P: maximal update parametrization

Maximal update parametrization (μ P) came out of this context as a response to both NTK and practical scaling problems:

- Key goal: define a way of parameterizing networks and scaling learning rates with width so that:
 - As width grows, the *effective* updates to activations/features remain $O(1)$.
 - Optimizer hyperparameters become *transferable* across model sizes.
 - The wide-limit behavior captures nontrivial feature learning, not just NTK-style lazy training.

Historically, μ P’s motivation is very close to the “max per-parameter / per-layer update” idea you already articulated for ℓ_∞ norms and signSGD, but at the level of *features* and *layers*, not individual parameters:

- NTK: keep parameters so close to initialization that features barely change → good theory, limited expressivity.
- μ P: keep **updates at the right scale** so features change meaningfully but don’t blow up as width grows → better alignment with practice and stable scaling across widths.

This is why your lecture notes’ focus on:

I actually learned a lot from this section, I never considered μ P as a response to the NTK infinite-width limit idea.

- norm-constrained steps,
- per-parameter/per-layer maximum updates,
- and the geometry defined by the optimizer,

is historically very aligned with μP : it's the same conceptual move, but now treated as a *principled scaling law* rather than a heuristic.

e) μP -aware optimizers (e.g., Muon)

Once you adopt μP (or related scaling ideas), you naturally start asking:

- Can we design optimizers whose behavior is *invariant* or *predictable* as we scale width?
- Can we make the **effective update size** (e.g., in activation space) behave like a controlled resource?

Muon-style optimizers live in that space:

- They treat the optimizer as part of the *parameterization* choice.
- They attempt to ensure that, across layers and widths, the effective step in some meaningful norm (often linked to activations or induced norms) stays controlled and comparable.
- Historically, this is a continuation of the same line from:
 - "steepest descent under a norm" (19th–20th c.),
 - to "adaptive per-coordinate norms" (Adagrad/Adam, 2010s),
 - to "structured/matrix norms" (K-FAC/Shampoo),
 - to "scaling laws and parameterization" (NTK, μP , 2018+),
 - and now "optimizers tailored to those parameterizations" (Muon and similar).

So the big historical picture tying your notes together is:

- Classical optimization (steepest descent under a norm, Newton, natural gradient).
- Per-coordinate and per-layer geometry (Adagrad/Adam → K-FAC/Shampoo).
- Initialization and signal-propagation theory (Xavier/He, mean-field).
- NTK: infinite-width lazy regime, kernel limit.
- μP : fix the *scaling* so updates remain meaningful and stable as width changes.
- μP -aware optimizers (Muon): treat the optimizer as an explicit part of the parameterization and scaling toolkit.

That completes the high-level historical arc behind the ideas in your lecture notes and finishes our planned 5-step lesson.

To check your understanding and help you cement this:

In your own words, what's the main reason people looked beyond NTK and started developing μP -style parameterizations—what problem were they trying to fix?



Another example of a summary question that checks for high-level understanding

1. https://en.wikipedia.org/wiki/History_of_artificial_neural_networks
2. <https://www.labellerr.com/blog/evolution-of-neural-networks-to-large-language-models/>

3. [https://en.wikipedia.org/wiki/Neural_network_\(machine_learning\)](https://en.wikipedia.org/wiki/Neural_network_(machine_learning))
4. https://en.wikipedia.org/wiki/History_of_artificial_intelligence
5. <https://en.wikipedia.org/wiki/Mind>
6. <https://en.wikipedia.org/wiki/Perceptron>
7. <https://en.wikipedia.org/wiki/Calculus>
8. https://en.wikipedia.org/wiki/Portal:History_of_science/Previous_pictures
9. <https://codewave.com/insights/development-of-neural-networks-history/>
10. <https://pub.towardsai.net/a-brief-history-of-neural-nets-472107bc2c9c>
11. <https://people.idsia.ch/~juergen/deep-learning-history.html>
12. <https://www.dataversity.net/articles/brief-history-deep-learning/>
13. [https://en.wikipedia.org/wiki/Neural_network_\(machine_learning\)](https://en.wikipedia.org/wiki/Neural_network_(machine_learning))
14. https://github.com/Ravoxsg/A_chronology_of_deep_learning
15. https://sebastianraschka.com/pdf/lecture-notes/stat479ss19/L02_dl-history_slides.pdf
16. <https://galileo-unbound.blog/2025/02/05/a-short-history-of-neural-networks/>



MuP allowed us to have more predictable optimizers whose behavior didn't depend on the many, arbitrary hyper parameters involved in deep learning

Not quite, but you're very close.

μ P doesn't remove hyperparameters or make optimizers "independent" of them. Instead, historically it was designed to fix two related problems:

- In standard parameterization, as you scale width,
 - features can freeze (NTK / lazy regime) or blow up, and
 - optimal hyperparameters (especially learning rate) shift a lot with width.
- μ P gives scaling rules so that:
 - feature updates stay order-one as width $\rightarrow \infty$ (so you get meaningful feature learning, not just NTK), and
 - the locations of good hyperparameters stay roughly *stable* as you change width, enabling hyperparameter transfer (μ Transfer).

Even when my answer is slightly off, Perplexity does a good job of making it more precise and correcting me.

So, more precise version of your sentence:

" μ P lets us choose parameterization and learning-rate scaling so that training dynamics and good hyperparameters are much more predictable as we scale model width, and the optimizer's effective behavior doesn't change wildly with width."

That's the core historical motivation: bridge infinite-width theory (NTK) and practical, feature-learning large models, while making tuning scalable.

We've now completed the 5-step lesson plan:

1. Steepest descent under norms,
2. $\ell^\infty \rightarrow$ sign updates,
3. Matrix geometry and preconditioning,
4. History from SGD $\rightarrow$ Adam $\rightarrow$ K-FAC/Shampoo,
5. NTK $\rightarrow$ μ P and μ P-aware optimizers.

To reinforce this:

Want to do a quick recap quiz (a few short questions) or some flashcard-style prompts on the historical milestones (NTK, Adagrad, K-FAC, μ P)?

*
**

1. <https://www.cerebras.ai/blog/the-practitioners-guide-to-the-maximal-update-parameterization>
2. <https://github.com/microsoft/mup>
3. <https://blog.eleuther.ai/mutransfer/>
4. <https://www.emergentmind.com/topics/maximal-update-parameterization-mup>
5. <https://aleph-alpha.com/in-awe-at-the-scale-of-these-tensors-a-gentle-introduction-to-unit-scaled-maximal-update-parametrization/>
6. <https://www.emergentmind.com/topics/maximal-update-parameterization-p>
7. <https://arxiv.org/html/2407.17465v3>
8. <https://kempnerinstitute.harvard.edu/research/deeper-learning/infinite-limits-of-neural-networks/>
9. <https://openreview.net/forum?id=fHt4Nau7FW>
10. https://proceedings.neurips.cc/paper_files/paper/2024/file/3b6aaffec941f98930753fa6d6de7263-Paper-Conference.pdf



Could you give a summary of the main ideas and historical progression?

Here's a compact, historically ordered summary that matches the ideas in your lecture notes (optimization, geometry, scaling), not a generic DL timeline.

1. Classical optimization → "choose a norm, get an update"

Key idea: Local linearization + norm constraint:

$$L(\theta + \Delta) \approx L(\theta) + \langle \nabla L, \Delta \rangle, \quad \min_{\|\Delta\| \leq \eta} \langle \nabla L, \Delta \rangle.$$

- In the 19th–20th century this is **steepest descent**: for a chosen norm, the solution gives the "steepest" direction.
- With Euclidean norm this recovers classical **gradient descent**.
- In the 1960s–1990s this view is generalized:
 - **Quadratic norms** $\|\Delta\|_A^2 = \Delta^\top A \Delta \rightarrow$ updates $\propto -A^{-1} \nabla L$: basis for **Newton's method** and later **natural gradient** (Amari, 1998).
 - This is the root of "optimization = pick a geometry (norm/metric), then take steepest descent in it."

This is the conceptual ancestor of *all* modern DL optimizers.

2. Norm choice → signSGD and per-parameter max updates

Switching the constraint to an ℓ_∞ **ball**:

$$\|\Delta\|_\infty \leq \eta \quad \Rightarrow \quad |\Delta_i| \leq \eta \quad \forall i$$

and minimizing a linear form coordinatewise gives:

$$\Delta_i^* = -\eta \operatorname{sign}(\nabla_i L).$$

- Historically, "steepest descent in ℓ_∞ " appears in mid-20th-century numerical optimization as one of the standard norm choices.
- In DL, this resurfaces as:
 - **signSGD** and low-precision/1-bit SGD (e.g. Seide et al. 2014, Bernstein et al. 2018).
 - Robust optimization and adversarial examples, where ℓ_∞ balls are standard perturbation sets.

- Conceptual shift: instead of controlling the *overall* step size, you cap the **maximum update per parameter**. That's the same philosophy behind "maximal update" thinking in later μP -style work.

So: just changing the norm from ℓ_2 to ℓ_∞ turns "follow the gradient" into "follow its sign."

3. From scalar steps $\rightarrow$ matrices: preconditioning, Newton, natural gradient

Using a quadratic norm with matrix A leads to updates:

$$\theta \leftarrow \theta - \eta A^{-1} \nabla L.$$

Historically:

- **Newton's method:** $A = \nabla^2 L$. Old (19th century) but central in numerical analysis: uses curvature to rescale directions.
- **Quasi-Newton** (BFGS, L-BFGS, 1970s–1980s): approximate $\nabla^2 L$ for cheaper updates.
- **Natural gradient** (Amari, 1998): $A = F$, the **Fisher information**; steepest descent in an information-geometric metric instead of raw parameter space.

Conceptual point:

Preconditioning = choose A to improve conditioning (make the Hessian more isotropic), so gradient steps progress similarly in all directions.

In DL this general idea later reappears as:

- "Diagonal" A : per-parameter learning rates (Adagrad/RMSprop/Adam).
- "Structured" A : layerwise approximate curvature (K-FAC, Shampoo).

4. SGD $\rightarrow$ Adagrad/RMSprop/Adam $\rightarrow$ K-FAC/Shampoo

4.1 Early DL training: SGD, momentum, and pain

1980s–2000s:

- Backprop (Rumelhart, Hinton, Williams 1986) + **SGD with momentum** became standard.
- Deep nets were hard to train due to **vanishing/exploding gradients** and bad conditioning.

4.2 Adaptive per-coordinate geometry

Key conceptual and historical jump: **each parameter should have its own effective step size**.

- **Adagrad** (Duchi et al., 2011):
 - Accumulate squared gradients per coordinate; step sizes shrink more where gradients are large.
 - Mathematically: a diagonal A_t built from gradient history.
- **RMSprop** (Tieleman & Hinton, 2012, lecture notes):

- Replace cumulative sum with an exponential moving average; suited for nonconvex problems like DL.
- **Adam** (Kingma & Ba, 2015):
 - Combine RMSprop's second-moment estimate with first-moment (momentum).
 - Became the de facto DL optimizer in the 2010s.

These are all **diagonal preconditioners**: A_t is diagonal, encoding a per-parameter geometry.

Parallel line: **initialization and variance preservation**

- **Glorot/Xavier init** (Glorot & Bengio, 2010): maintain activation variance across layers.
- **He init** (He et al., 2015): tuned for ReLU.

Both reflect the principle: control norms/variances of signals and gradients layerwise, so optimization is better conditioned.

4.3 From diagonal to matrix: K-FAC and Shampoo

Once diagonal methods were standard, the next question was: can we get closer to full curvature, but still scale?

- **K-FAC** (Martens & Grosse, 2015):
 - Approximate the Fisher/Hessian per layer as a Kronecker product of two smaller matrices.
 - Efficiently inverts this structured A to get a natural-gradient-like preconditioner.
- **Shampoo** (Gupta et al., 2018):
 - For each weight tensor, maintain one matrix per tensor mode (e.g., rows and columns for a matrix).
 - Precondition by inverse matrix powers along each mode: a scalable approximation to full second-order preconditioning.

These introduce **matrix/induced norms** per layer/tensor: you are no longer working in a scalar or diagonal geometry, but in an *operator geometry* that reflects the structure of weights.

Historically, that's the line:

Classical operator norms & spectral norms → matrix-geometric optimizers → K-FAC/Shampoo-type methods.

5. NTK, lazy training, and the emergence of μP -style scaling

5.1 NTK: wide nets $\approx$ kernel methods

Around 2018:

- **Neural Tangent Kernel (NTK)** work (e.g., Jacot et al., 2018) showed:
 - In the infinite-width limit, with small learning rates and standard init,
 - parameters barely move, and
 - training is equivalent to kernel regression with a fixed kernel (the NTK of the architecture).

Conceptually, in this regime:

- Training is **“lazy”**: networks stay near initialization, local linearization is globally valid.
- The function evolves linearly in parameters; theory from kernel methods applies.

This fits perfectly with the local-linear/steepest-descent view in your notes—but in a regime that’s arguably *too* linear for the rich feature learning we see in practice.

5.2 Why NTK wasn’t enough

Empirically:

- Very wide nets in NTK-like scaling can underperform practical finite-width nets.
- Real nets often move far from initialization and **change their features** significantly.
- Also, optimal learning rates and other hyperparameters often change with width; tuning does *not* transfer easily across scales.

So theory (NTK, lazy regime) and practice (feature-learning, large updates) were misaligned.

5.3 μ P: maximal update parametrization

μ P (maximal update parametrization) was developed to fix this mismatch:

- It prescribes how to scale **parameters and learning rates with width** so that:
 - Feature updates stay $O(1)$ as width increases (nontrivial feature learning in the limit).
 - “Good” hyperparameters tend to **transfer** across widths (μ Transfer).

Historically, μ P generalizes an idea you already saw in simpler form:

- Instead of just constraining parameter-norm or max per-parameter update (like ℓ_∞),
- μ P constrains **effective updates at the level of activations/features and layers**, via careful scaling rules.

So μ P is a response to NTK: keep the benefits of wide limits, but avoid being stuck in the fully lazy regime and get stable, predictable scaling.

5.4 μ P-aware optimizers (e.g., Muon)

Once we treat parameterization and scaling as first-class, the optimizer itself becomes part of that design:

- μ P-aware optimizers (like **Muon**) aim to keep **effective update sizes** in meaningful norms roughly invariant across layers and widths.
- This continues the same line:
 - “pick a norm / metric → you get a steepest descent rule,”
 - but now tuned so that as models scale, the **geometry and update magnitudes behave predictably**.

Historically, this is the culmination of:

- Classical steepest descent under norms →
- per-coordinate adaptivity →
- matrix/structured geometry →
- infinite-width limits and NTK →
- μ P scaling & μ P-aware optimizers.

Very briefly, putting it all in one sentence:

The story runs from classical steepest descent under a norm, through per-parameter and matrix preconditioning (SGD → Adam → K-FAC/Shampoo), into infinite-width theory (NTK) and finally to μ P-style parameterizations and μ P-aware optimizers that explicitly control geometry and update scales as networks grow.

To check your own grasp: if you had to pick *one* turning point paper (or idea) that you think changed how people viewed optimization geometry in deep learning—what would you pick and why?

✱

The final summary seems to generally capture all the ideas that were discussed above. Overall, this seems fairly open-ended and the student could ask any question next depending on what they wanted to learn.

1. <https://developer.nvidia.com/blog/deep-learning-nutshell-history-training/>
2. https://sebastianraschka.com/pdf/lecture-notes/stat453ss21/L02_dl-history_slides.pdf
3. https://en.wikipedia.org/wiki/Deep_learning
4. <https://www.dataversity.net/articles/brief-history-deep-learning/>
5. https://en.wikipedia.org/wiki/Deep_learning
6. <https://getthematic.com/insights/what-is-deep-learning>
7. <https://people.idsia.ch/~juergen/deep-learning-history.html>
8. <https://www.flagshipioneering.com/timelines/a-timeline-of-deep-learning>
9. https://en.wikipedia.org/wiki/Timeline_of_machine_learning
10. https://github.com/Ravoxsg/A_chronology_of_deep_learning

11. <https://towardsdatascience.com/a-brief-history-of-deep-learning-frameworks-8debf3ba6607/>